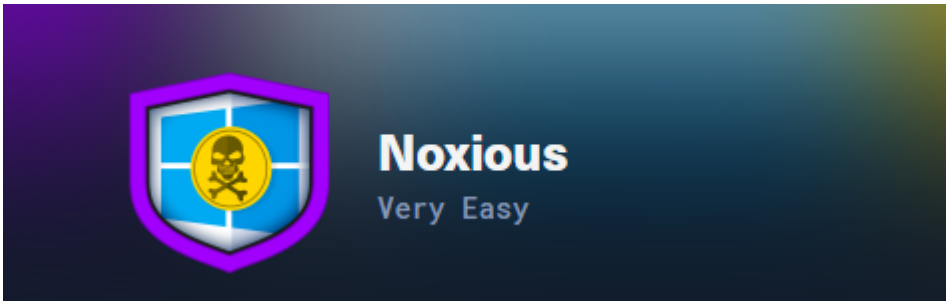


Senario ➖ The IDS device alerted us to a possible rogue device in the internal Active Directory network. The Intrusion Detection System also indicated signs of LLMNR traffic, which is unusual. It is suspected that an LLMNR poisoning attack occurred. The LLMNR traffic was directed towards Forela-WKstn002, which has the IP address 172.17.79.136. A limited packet capture from the surrounding time is provided to you, our Network Forensics expert. Since this occurred in the Active Directory VLAN, it is suggested that we perform network threat hunting with the Active Directory attack vector in mind, specifically focusing on LLMNR poisoning.

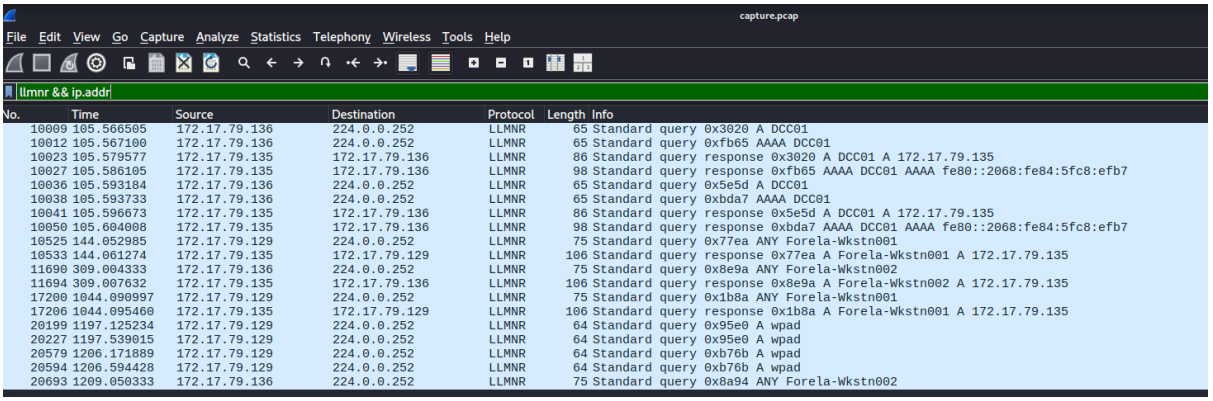
Analysis ➖ Poisoned Traffic

The Provided File is “Capture.pcap” which Contains Captured Network Traffic that we are Going To analyze , saving the Hash checksum is essential to verify the file integrity before opening it

```
┌──(kali㉿kali)-[~/Htb/Noxious]
└─$ md5sum capture.pcap
dfaa529285a59d0ccd299355f0169055  capture.pcap
```

Now I am Ready To open the Capture File in Wireshark and start Analyzing …

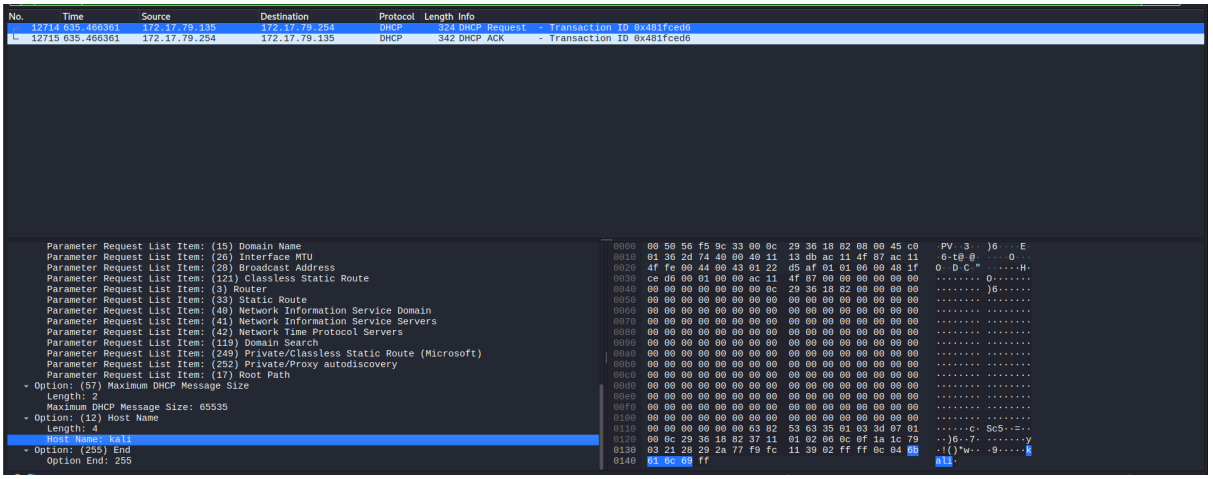
The security team suspects LLMNR poisoning which is usually done by attackers to gain foothold in the network by getting NTLM hashes by poisoning DNS fallback protocols like LLMNR/NBT-NS and others . Filter Traffic by protocol in the search bar



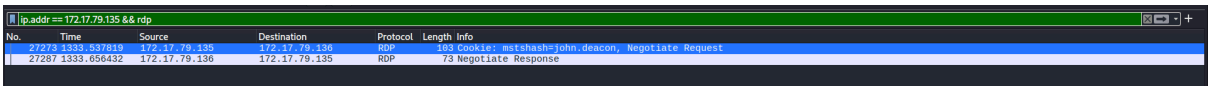
The Way LLMNR works usually when a user types a wrong host mistyping it or query a host that is no longer exists so instead of DNS it starts to send LLMNR request by

casting in the Network Looking for any Response , here where attackers Poison this cast and respond as if they were Legit , this happens first in Layer 2 between Mac addresses thats why i put in my filter “ip.addr” to avoid the Noisy mac address traffics . After the attacker responds, the victim device starts to authenticate to it, sending an NTLM hash where the attacker can crack it offline or use it in NTLM-relaying attacks .

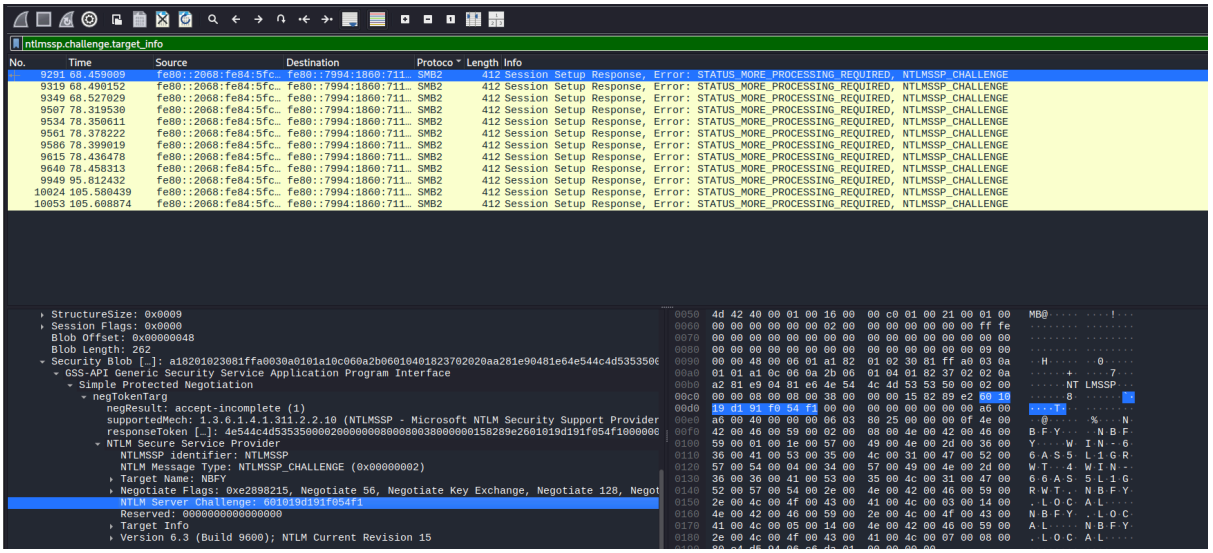
So the attacker is the one who is responding to these queries. His IP is **172.17.79.135** and it seems he collected 2 NTLM hashes of the Devices with the IPs 172.17.79.129 & 172.17.79.136 . For More Informations Regards the Host i filtered upon the Dynamic Host Configuration Protocol(DHCP)



So the attacker seems to run VMWare Workstation, specifically Kali Linux with the hostname of **Kali** .



To check for a successful crack of a user password I checked the Remote Access Management Protocol RDP and found a RDP authentication token from the attacker machine for the user **john.deacon** . The First LLMNR Poisoned victim happened at **2024-06-24 11:18:30** according to our previous analysis for LLMNR Traffic with the ip address of 172.17.79.136 so to find the root cause of poisoning i analyzed the First LLMNR poisoned response which reveals that the victim typed **DCC01** instead of DC01 which is the domain controller name that exposed him to this attack .



To find The Credentials been cracked i filtered upon “ntlmssp.challenge.target_info” to analyze NTLM authentication and start analyzing the First negotiation response to find the server challenge Value under > security Blob > GSS API > Simple Protected Negotiation > negTokenTarg > NTLM Server Challenge with value of **601019d191f054f1**

Now To Find the NTProofString Value to reconstruct the NTLM hash i filtered upon “ntlmssp.auth.ntresponse” and found the latest response going under > Session setup > Security Blob > GSS-API > negTokenTarg > NTLM Secure Service Provider > NTLM Response > NTProofStr with value of **c0cc803a6d9fb5a9082253a04dbd4cd4**

I will reconstruct the NTLM Hash and try cracking it myself to find out its complexity and the estimation time the attacker took to crack it .
The NTLM hash structure is
User::Domain:ServerChallenge:NTProofStr:NTLMV2Response

The NTLMV2Response is just the full hash without the NTProofStr which is the first 32 characters of it . The NTLMresponse is

```
c0cc803a6d9fb5a9082253a04dbd4cd401010000000000080e4d59406c6da01cc3d
cfc0de9b5f2600000000020008004e0042004600590001001e00570049004e002d00
3600360041005300350004c003100470052005700540004003400570049004e002d00
3600360041005300350004c003100470052005700540002e004e004200460059002e00
4c004f00430041004c00030014004e004200460059002e004c004f00430041004c00
050014004e004200460059002e004c004f00430041004c000700080080e4d59406c6
da0106000400020000000800300030000000000000000000000000200000eb2ecbc5
200a40b89ad5831abf821f4f20a2c7f352283a35600377e1f294f1c90a0010000000
0000000000000000000000000000900140063006900660073002f00440043004300
300031000000000000000000
```

The final hash would be this

```
┌──(kali㉿kali)-[~/Htb/Noxious]
└─$ cat hash.txt
john.deacon::FORELA:601019d191f054f1:c0cc803a6d9fb5a9082253a04dbd4
cd4:0101000000000000080e4d59406c6da01cc3dcfc0de9b5f2600000000020008
004e0042004600590001001e00570049004e002d0036003600410053003500 4c00
3100470052005700540004003400570049004e002d0036003600410053003500 4c
0031004700520057005400040034005700490 04e002d0036003600410053003500 4c
00310047005200570054002e004e004200460059002e004c004f00430041004c00
030014004e004200460059002e004c004f00430041004c00050014004e00420046
0059002e004c004f00430041004c000700080080e4d59406c6da01060004000200
0000080030003000000000000000000000000200000eb2ecbc5200a40b89ad583
1abf821f4f20a2c7f352283a35600377e1f294f1c90a0010000000000000000000
0000000000000000090014006300690066007300 2f00440043004300030000310
00000000000000
```

Now i am ready To crack

```
┌──(kali㉿kali)-[~/Htb/Noxious]
└─$ hashcat -m 5600 hash.txt rockyou.txt
hashcat (v7.1.2) starting


Dictionary cache built:
* Filename..: rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 0 secs

JOHN.DEACON::FORELA:601019d191f054f1:c0cc803a6d9fb5a9082253a04dbd4
cd4:0101000000000000080e4d59406c6da01cc3dcfc0de9b5f2600000000020008
004e0042004600590001001e00570049004e002d0036003600410053003500 4c00
3100470052005700540004003400570049004e002d0036003600410053003500 4c
0031004700520057005400040034005700490 04e002d0036003600410053003500 4c
00310047005200570054002e004e004200460059002e004c004f00430041004c00
030014004e004200460059002e004c004f00430041004c00050014004e00420046
0059002e004c004f00430041004c000700080080e4d59406c6da01060004000200
0000080030003000000000000000000000000200000eb2ecbc5200a40b89ad583
1abf821f4f20a2c7f352283a35600377e1f294f1c90a0010000000000000000000
0000000000000000090014006300690066007300 2f00440043004300030000310
00000000000000:NotMyPassword0k?
```

```
Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 5600 (NetNTLMv2)
Hash.Target......:
JOHN.DEACON::FORELA:601019d191f054f1:c0cc803a6d9fb5...000000
Time.Started.....: Thu Oct 15 06:49:39 2026 (4 secs)
Time.Estimated...: Thu Oct 15 06:49:43 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  2971.2 kH/s (1.41ms) @ Accel:1024 Loops:1
Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%)
Digests (new)
Progress.........: 10780672/14344385 (75.16%)
Rejected.........: 0/10780672 (0.00%)
Restore.Point....: 10772480/14344385 (75.10%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
```

It Only took 4 seconds To crack with a pretty much easy-to-guess Password of **NotMyPassword0k?** . So all this attack chain started when the victim tried to access the share **\\DC01\DC-Confidential** and mistyped the hostname DC01 with DCC01 which made him vulnerable to LLMMNR poisoning Attack .

➖Remediation

To remediate LLMNR and NBT-NS poisoning, administrators must explicitly disable these legacy name-resolution protocols across the network via Group Policy.
Enforcing SMB Signing across all domain shares prevents attackers from successfully relaying captured authentication traffic to other systems.
Because captured Net-NTLMv2 hashes are vulnerable to offline dictionary attacks using tools like Hashcat, organizations must mandate strict password complexity standards.Implementing high minimum password lengths renders standard wordlists and brute-force cracking completely ineffective.Transitioning core authentication workflows toward modern protocols like Kerberos ultimately eliminates the risks associated with challenge-response hash capture.